



IIC2343 - Arquitectura de Computadores (II/2026)

Ayudantía 5

Ayudantes: Ignacio Gajardo (gajardo.ignacio@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl),
Nicolás Romo Aubele (nroma@uc.cl), Carlos Canales (carlos.canales@estudiante.uc.cl)

Pregunta 1: Preguntas Conceptuales:

a) En el computador básico del curso, la memoria de *stack* se utiliza para soportar la ejecución de subrutinas. Describa cómo está organizada esta memoria y el rol del registro SP (*Stack Pointer*). Luego, explique por qué es fundamental inicializar correctamente el SP y analice las consecuencias de una inicialización incorrecta. Finalmente, discuta cómo estos problemas se ven acentuados en presencia de llamadas recursivas y explique en qué consiste un *stack overflow* en este contexto.

b) Considera el siguiente código en *assembly* del computador básico. Explique el funcionamiento y propósito en el código de las instrucciones POP, PUSH, CALL y RET; y cuál sería el efecto si no hubiera una instrucción PUSH A justo antes, ni una instrucción POP A justo después de la llamada a la función calc.

```
CODE:
    MOV A, 5
    MOV B, 0

loop:
    CMP A, 0
    JEQ end
    PUSH A
    CALL calc
    POP A
    SUB A, 1
    JMP loop

calc:
    ADD B, A
    RET

end:
```

Solución: a) En el computador básico, la memoria de *stack* corresponde a una región de la memoria de datos ubicada en el segmento inferior, utilizada para almacenar información temporal como direcciones de retorno y valores de registros. El acceso se realiza mediante el registro **SP** (*Stack Pointer*), que se inicializa en la última dirección de memoria (por ejemplo, 255 en una memoria de 8 *bits*). A medida que se ejecutan instrucciones como **PUSH** o **CALL**, el *stack* crece hacia direcciones menores, decrementando el valor de **SP**.

Inicializar correctamente el **SP** es fundamental para asegurar que el *stack* tenga suficiente espacio para crecer sin interferir con las variables del programa, manteniendo separadas ambas regiones de memoria.

Si el **SP** se inicializa incorrectamente, por ejemplo en la dirección 0, o en la 64, pueden producirse errores graves. Ya que el *stack* podría sobrescribir variables del programa, generando corrupción de datos, y las instrucciones **CALL** y **RET** pueden utilizar direcciones de retorno incorrectas, afectando el flujo de ejecución.

Este problema se vuelve aún más crítico en presencia de recursión, donde múltiples llamadas a una misma subrutina se acumulan en el *stack*. Cada llamada recursiva requiere almacenar su propia dirección de retorno y posibles datos adicionales, por lo que el uso del *stack* crece rápidamente. Si el espacio disponible es insuficiente —ya sea por una mala inicialización del **SP** o por una gran profundidad de recursión— puede ocurrir *stack overflow*, donde el *stack* colisiona con otras regiones de memoria. Esto provoca comportamiento incorrecto, pérdida de información o incluso la detención del programa.

b) El código ejecuta un ciclo iterativo controlado por el registro **A**, el cual inicializa en 5 y disminuye en 1 en cada iteración mediante la instrucción **SUB A, 1**. La condición de término se evalúa al inicio del ciclo con **CMP A, 0**, saltando al final del programa mediante **JEQ end** cuando el contador llega a cero.

Dentro del ciclo, se invoca a la subrutina **calc**, la cual acumula el valor actual de **A** en el registro **B** a través de la instrucción **ADD B, A**. Como esto se repite para cada iteración, el registro **B** terminará almacenando la sumatoria de los valores descendentes de **A** ($5 + 4 + 3 + 2 + 1 = 15$).

Es esencial el uso de la instrucción **CALL** para iniciar la subrutina y **RET** para poder retornar correctamente a la dirección guardada en el *stack*.

El uso de las instrucciones **PUSH A** y **POP A** es fundamental para resguardar el valor del contador del ciclo antes de la llamada a la subrutina con **CALL calc**. Al guardar el valor de **A** en el *stack* y restaurarlo al volver, se protege la variable de control del ciclo, garantizando que este finalice correctamente y no se vea afectado por posibles modificaciones locales dentro de la función.

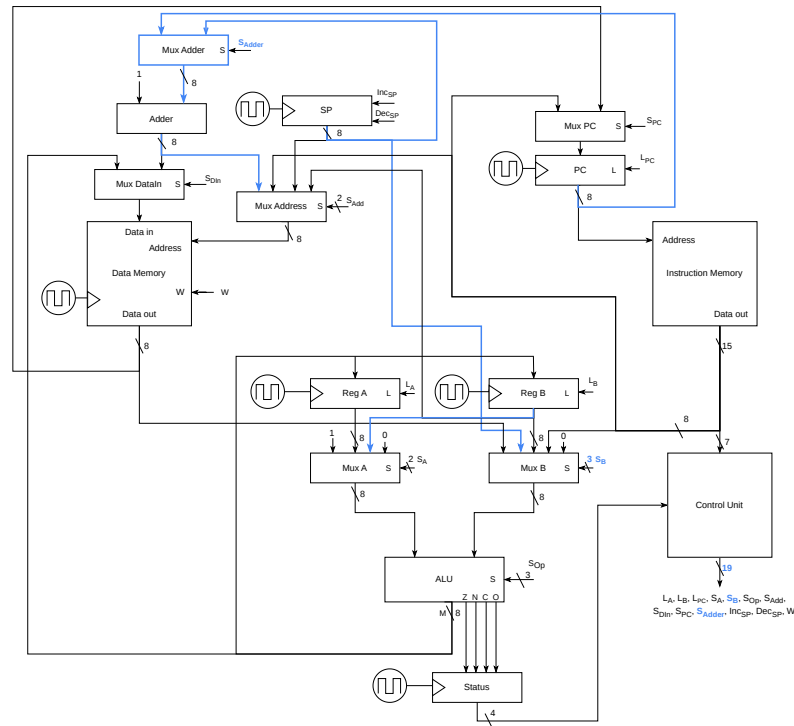
Al finalizar la ejecución del programa, el registro **A** alcanza la condición de término quedando en 0, y el registro **B** termina con un valor de 15.

Pregunta 2: Computador básico con subrutinas (I2 2026-1)

En los ítems (a), (b) y (c), se le describirán instrucciones que deberá implementar en el computador básico con subrutinas a partir del diagrama adjunto. Para cada una de ellas, debe incluir la combinación **completa** de señales que las ejecutan. En las señales de carga/escritura/incremento/decremento, debe indicar si se activan (1) o no (0); en las señales de selección, debe indicar el **nombre** de la entrada escogida (“-” si no afecta). Debe realizar todas las modificaciones de *hardware* necesarias en un solo diagrama. Además, puede implementar instrucciones sin modificar el *hardware* si es que la arquitectura se lo permite, indicando la forma de hacerlo a partir de las señales de control. **Cada instrucción debe ejecutarse en un ciclo.**

- (a) **PUSHRW A**. Sobreescribe el tope del *stack* con el valor del registro A sin modificar el *stack pointer*.
- (b) **CMP B, SP**. Computa la operación B - SP sin actualizar los registros.
- (c) **RET1**. Salta a la dirección de retorno almacenada en el tope del *stack* **en un solo ciclo**. En otras palabras, es la versión óptima de **RET** al realizar la misma operación en un ciclo y no en dos.

Solución: A continuación, se muestra el diagrama con las conexiones necesarias para ejecutar todas las instrucciones solicitadas, incluyendo una descripción de cada conexión añadida y lo que habilita.



- Se modifica la entrada del *Adder* que computa PC+1 para añadir un multiplexor *Mux Adder* con señal de selección S_{Adder} . Este selecciona entre el valor de PC y el de SP para realizar la suma, lo que da la posibilidad de computar PC+1 o SP+1.
- Se conecta la salida del *Adder* con la entrada restante del *Mux Address*, permitiendo seleccionar PC+1 o SP+1 (el que nos interesa) como dirección de la memoria de datos.
- Se conecta la salida de B con una entrada de *Mux A* y la salida de SP con una entrada de *Mux B*, de forma que puedan seleccionarse como primer y segundo operando de la ALU respectivamente. Esto requiere incrementar a 3 bits la señal S_B .

A continuación, la tabla de señales resultante:

Instrucción	L _A	L _B	L _{PC}	Inc _{SP}	Dec _{SP}	W	S _A	S _B	S _{Op}	S _{Add}	S _{DIn}	S _{PC}	S _{Adder}
PUSHRW A	0	0	0	0	0	1	A	ZERO	ADD	ADDER	ALU	-	SP
CMP B, SP	0	0	0	0	0	0	B	SP	SUB	-	-	-	-
RET1	0	0	1	1	0	0	-	-	-	ADDER	-	DOUT	SP

d) A continuación, se adjunta un fragmento de código escrito en el Assembly del computador básico que incorpora las instrucciones implementadas en los ítems anteriores. Indique los valores de los registros A y B al finalizar la ejecución del código.

(a)

```
DATA:
    n    5
    res  0
CODE:
    MOV A,(n)      ; Dir. memoria 0x00
    CALL fun       ; Dir. memoria 0x01
    JMP end        ; Dir. memoria 0x02
fun:
    CMP A,0        ; Dir. memoria 0x03
    JLE fun_end    ; Dir. memoria 0x04
    PUSH A         ; Dir. memoria 0x05
    MOV B,A        ; Dir. memoria 0x06
    AND A,1        ; Dir. memoria 0x07
    JLE rewrite    ; Dir. memoria 0x08
continue:
    MOV A,B        ; Dir. memoria 0x09
    SUB A,1        ; Dir. memoria 0x0A
    JMP fun        ; Dir. memoria 0x0B
rewrite:
    MOV A,0        ; Dir. memoria 0x0C
    PUSH RW A      ; Dir. memoria 0x0D
    JMP continue   ; Dir. memoria 0x0E
fun_end:
    MOV B,254      ; Dir. memoria 0x0F
loop:
    CMP B,SP       ; Dir. memoria 0x10
    JE end_loop    ; Dir. memoria 0x11
    POP A          ; Dir. memoria 0x12-0x13
    ADD A,(res)    ; Dir. memoria 0x14
    MOV (res),A    ; Dir. memoria 0x15
    JMP loop       ; Dir. memoria 0x16
end_loop:
    RET1          ; Dir. memoria 0x17
end:
```

Solución: El fragmento de código hace lo siguiente: (i) llama a la subrutina **fun** y *pushea* los valores **n**, **n-1**, **n-2**, ..., 1; (ii) al *pushear*, revisa si el valor escrito es par –**AND A,1** < 1– y, en dicho caso, lo **sobreescribe por cero** –**PUSHRW A** en 0x0D–; (iii) al terminar de *pushear* y sobreescribir los pares, descarta el contenido del *stack* y acumula la suma de los elementos en **res** a través de **A**; (iv) una vez que los revisa todos –**SP** = 254, siendo el tope del *stack* la dirección de retorno de **fun**–, retorna y termina el programa. Por ende, **A** es igual a 9 (5 + 3 + 1, los impares del rango [1, 5]), mientras que **B** es igual a 254, ya que la última vez que actualiza su valor es con **MOV B,254** en 0x0F. A continuación, se muestra el resultado de la ejecución del fragmento:

```
DATA:
    n      5
    res    0
CODE:
    MOV A,(n)      ; A = 5
    CALL fun       ; MEM[255] = 0x02
    JMP end        ; Termina el programa
fun:
    CMP A,0        ; if A <= 0
    JLE fun_end    ; -> Salta a fun_end
    PUSH A         ; MEM[254] = 5, MEM[253] = 4, ..., MEM[250] = 1 y SP = 249
    MOV B,A        ; B = 5, 4, 3, 2, 1
    AND A,1        ; 5 AND 1 = 1, 4 AND 1 = 0, 3 AND 1 = 1, 2 AND 1 = 0, 1 AND 1 = 1
    JLE rewrite    ; Si A es par, salta a rewrite
    continue:      ; e.o.c. es impar y continua sin reescribir
        MOV A,B    ; A = 5, 4, 3, 2, 1
        SUB A,1    ; A = 4, 3, 2, 1, 0
        JMP fun    ; Repetir hasta que A == 0
    rewrite:
        MOV A,0    ; A = 0
        PUSH RW A  ; MEM[253] = 0, MEM[251] = 0
        JMP continue ; Salta a continue
    fun_end:
        MOV B,254  ; B = 254
    loop:
        CMP B,SP   ; if SP == B (se revisaron los elementos pusheados)
        JEQ end_loop ; -> Salta a end_loop
        POP A      ; A = MEM[250] y SP = 250, ..., A = MEM[254] y SP = 254
        ADD A,(res) ; A += res (acumula suma en res)
        MOV (res),A ; res = A (actualiza res)
        JMP loop   ; Repetir hasta que SP == 254
    end_loop:
        RETI       ; PC = MEM[255] = 0x02 (retorna a JMP end en un ciclo)
end:      ; A = res = 5 + 3 + 1 = 9; B = 254 (última actualización con MOV B,254)
```

Computador básico con subrutinas

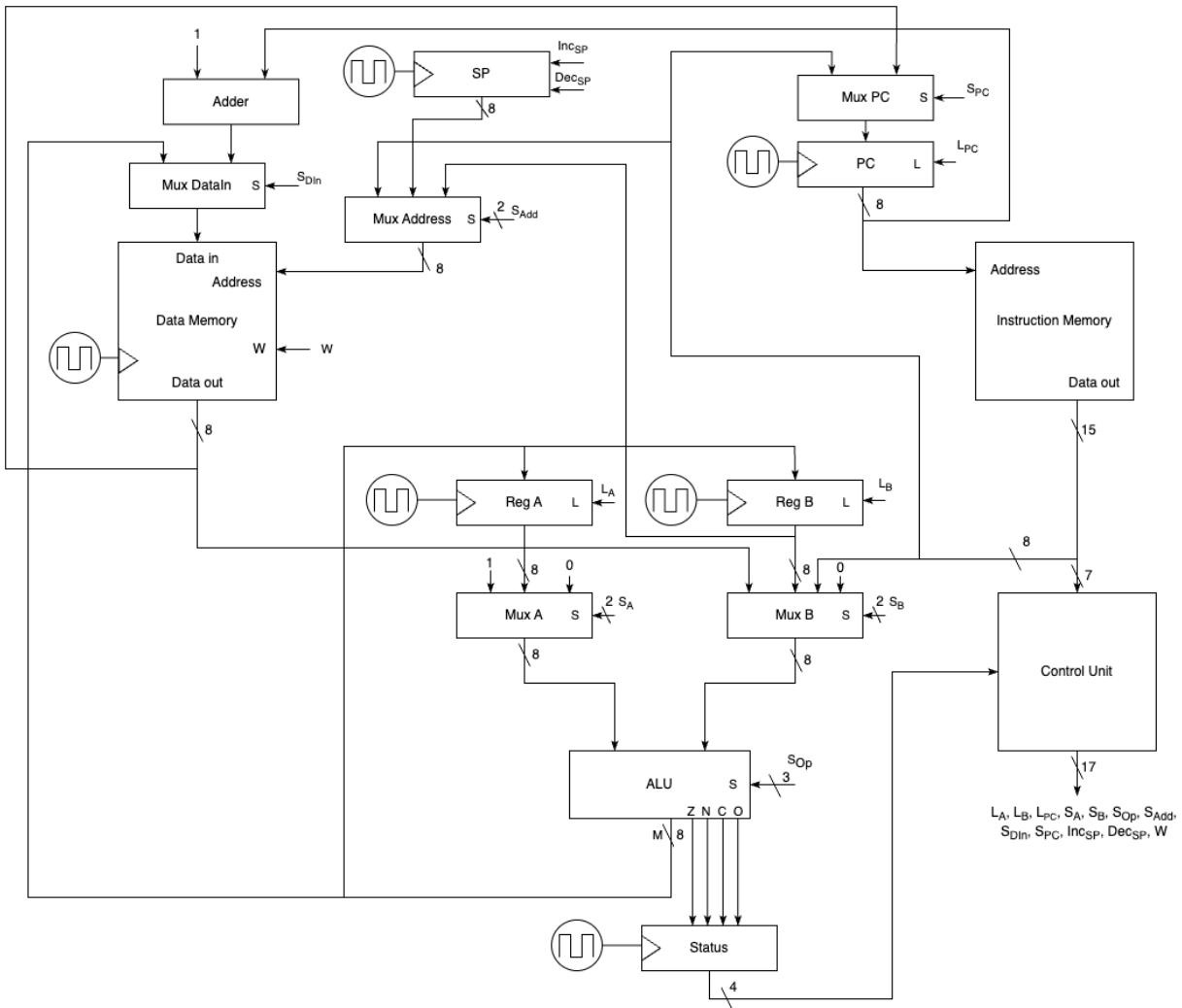


Tabla de instrucciones

Instrucción	Operandos	Opcode	Condición	Lpc	LA	LB	SA0,1	SB0,1	SP0,1,2	SAdd0,1	SDin	SPC	W	IncSP	DecSP
MOV	A,B	0000000		0	1	0	ZERO	B	ADD	-	-	-	0	0	0
	B,A	0000001		0	0	1	A	ZERO	ADD	-	-	-	0	0	0
	A,Lit	0000010		0	1	0	ZERO	LIT	ADD	-	-	-	0	0	0
	B,Lit	0000011		0	0	1	ZERO	LIT	ADD	-	-	-	0	0	0
	A,(Dir)	0000100		0	1	0	ZERO	DOUT	ADD	LIT	-	-	0	0	0
	B,(Dir)	0000101		0	0	1	ZERO	DOUT	ADD	LIT	-	-	0	0	0
	(Dir),A	0000110		0	0	0	A	ZERO	ADD	LIT	ALU	-	1	0	0
	(Dir),B	0000111		0	0	0	ZERO	B	ADD	LIT	ALU	-	1	0	0
	A,(B)	0001000		0	1	0	ZERO	DOUT	ADD	B	-	-	0	0	0
ADD	B,(B)	0001001		0	0	1	ZERO	DOUT	ADD	B	-	-	0	0	0
	(B),A	0001010		0	1	0	A	ZERO	ADD	B	ALU	-	1	0	0
	A,B	0001011		0	1	0	A	B	ADD	-	-	-	0	0	0
	B,A	0001100		0	0	1	A	B	ADD	-	-	-	0	0	0
	A,Lit	0001101		0	1	0	A	LIT	ADD	-	-	-	0	0	0
	A,(Dir)	0001110		0	1	0	A	DOUT	ADD	LIT	-	-	0	0	0
	A,(B)	0001111		0	1	0	A	DOUT	ADD	B	-	-	0	0	0
	(Dir)	0010000		0	0	0	A	B	ADD	LIT	ALU	-	1	0	0
	(Dir)	0010000		0	0	0	A	B	ADD	LIT	ALU	-	1	0	0
SUB	A,B	0010001		0	1	0	A	B	SUB	-	-	-	0	0	0
	B,A	0010010		0	0	1	A	B	SUB	-	-	-	0	0	0
	A,Lit	0010011		0	1	0	A	LIT	SUB	-	-	-	0	0	0
	A,(Dir)	0010100		0	1	0	A	DOUT	SUB	LIT	-	-	0	0	0
	A,(B)	0010101		0	1	0	A	DOUT	SUB	B	-	-	0	0	0
	(Dir)	0010110		0	0	0	A	B	SUB	LIT	ALU	-	1	0	0
	(Dir)	0010110		0	0	0	A	B	SUB	LIT	ALU	-	1	0	0
	A,B	0010111		0	1	0	A	B	AND	-	-	-	0	0	0
	B,A	0011000		0	0	1	A	B	AND	-	-	-	0	0	0
AND	A,Lit	0011001		0	1	0	A	LIT	AND	-	-	-	0	0	0
	A,(Dir)	0011010		0	1	0	A	DOUT	AND	LIT	-	-	0	0	0
	A,(B)	0011011		0	1	0	A	DOUT	AND	B	-	-	0	0	0
	(Dir)	0011100		0	0	0	A	B	AND	LIT	ALU	-	1	0	0
	(Dir)	0011100		0	0	0	A	B	AND	LIT	ALU	-	1	0	0
	A,B	0011101		0	1	0	A	B	OR	-	-	-	0	0	0
	B,A	0011110		0	0	1	A	B	OR	-	-	-	0	0	0
	A,Lit	0011111		0	1	0	A	LIT	OR	-	-	-	0	0	0
	A,(Dir)	0100000		0	1	0	A	DOUT	OR	LIT	-	-	0	0	0
OR	A,(B)	0100001		0	1	0	A	DOUT	OR	B	-	-	0	0	0
	(Dir)	0100010		0	0	0	A	B	OR	LIT	ALU	-	1	0	0
	(Dir)	0100010		0	0	0	A	B	OR	LIT	ALU	-	1	0	0
	A,A	0100011		0	1	0	A	-	NOT	-	-	-	0	0	0
	B,A	0100100		0	0	1	A	-	NOT	-	-	-	0	0	0
	(Dir)	0100101		0	0	0	A	B	NOT	LIT	ALU	-	1	0	0
	(Dir)	0100101		0	0	0	A	B	NOT	LIT	ALU	-	1	0	0
	A,B	0100110		0	1	0	A	B	XOR	-	-	-	0	0	0
	B,A	0100111		0	0	1	A	B	XOR	-	-	-	0	0	0
XOR	A,Lit	0101000		0	1	0	A	LIT	XOR	-	-	-	0	0	0
	A,(Dir)	0101001		0	1	0	A	DOUT	XOR	LIT	-	-	0	0	0
	A,(B)	0101010		0	1	0	A	DOUT	XOR	B	-	-	0	0	0
	(Dir)	0101011		0	0	0	A	B	XOR	LIT	ALU	-	1	0	0
	(Dir)	0101011		0	0	0	A	B	XOR	LIT	ALU	-	1	0	0
	A,A	0101100		0	1	0	A	-	SHL	-	-	-	0	0	0
	B,A	0101101		0	0	1	A	-	SHL	-	-	-	0	0	0
	(Dir)	0101110		0	0	0	A	B	SHL	LIT	ALU	-	1	0	0
	(Dir)	0101110		0	0	0	A	B	SHL	LIT	ALU	-	1	0	0
SHL	A,A	0101111		0	1	0	A	-	SHR	-	-	-	0	0	0
	B,A	0110000		0	0	1	A	-	SHR	-	-	-	0	0	0
	(Dir)	0110001		0	0	0	A	B	SHR	LIT	ALU	-	1	0	0
	(Dir)	0110001		0	0	0	A	B	SHR	LIT	ALU	-	1	0	0
	A,A	0110000		0	0	1	A	-	SHR	-	-	-	0	0	0
	B,A	0110001		0	0	1	A	-	SHR	-	-	-	0	0	0
	(Dir)	0110001		0	0	0	A	B	SHR	LIT	ALU	-	1	0	0
	(Dir)	0110001		0	0	0	A	B	SHR	LIT	ALU	-	1	0	0
	A,A	0110010		0	0	1	ONE	B	ADD	-	-	-	0	0	0
INC	B	0110010		0	0	1	ONE	B	ADD	-	-	-	0	0	0
	A,B	0110101		0	0	0	A	B	SUB	-	-	-	0	0	0
	A,Lit	0110110		0	0	0	A	LIT	SUB	-	-	-	0	0	0
	A,Lit	0110110		0	0	0	A	LIT	SUB	-	-	-	0	0	0
	Dir	0111001		1	0	0	-	-	-	-	-	LIT	0	0	0
	Dir	0111010	Z=1	1	0	0	-	-	-	-	-	LIT	0	0	0
	Dir	0111011	Z=0	1	0	0	-	-	-	-	-	LIT	0	0	0
	Dir	0111100	N=0 y Z=0	1	0	0	-	-	-	-	-	LIT	0	0	0
	Dir	0111101	N=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JMP	Dir	0111110	N=0	1	0	0	-	-	-	-	-	LIT	0	0	0
	Dir	0111111	N=1 o Z=1	1	0	0	-	-	-	-	-	LIT	0	0	0
	Dir	1000000	C=1	1	0	0	-	-	-	-	-	LIT	0	0	0
	Dir	1000001	V=1	1	0	0	-	-	-	-	-	LIT	0	0	0
	Dir	1000001	V=1	1	0	0	-	-	-	-	-	LIT	0	0	0
	Dir	1000010		1	0	0	-	-	-	SP	PC	LIT	1	0	1
	Dir	1000011		0	0	0	-	-	-	-	-	-	0	1	0
	Dir	1000100		1	0	0	-	-	-	SP	-	DOUT	0	0	0
	Dir	1000101		0	0	0	A	ZERO	ADD	SP	ALU	-	1	0	1
PUSH	A	1000110		0	0	0	ZERO	B	ADD	SP	ALU	-	1	0	1
	B	1000111		0	0	0	-	-	-	-	-	-	0	1	0
	A	1000011		0	0	0	-	-	-	-	-	-	0	1	0
	A	1000111		0	1	0	ZERO	DOUT	ADD	SP	-	-	0	0	0
	B	1000011		0	0	0	-	-	-	-	-	-	0	1	0
	B	1001000		0	0	1	ZERO	DOUT	ADD	SP	-	-	0	0	0
	B	1001000		0	0	1	ZERO	DOUT	ADD	SP	-	-	0	0	0
	B	1001000		0	0	1	ZERO	DOUT	ADD	SP	-	-	0	0	0
	B	1001001		0	0	0	-	-	-	-	-	-	0	0	0
POP	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
NOP	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0
	A	1001001		0	0	0	-	-	-	-	-	-	0	0	0

Tabla 1: ISA del computador básico.

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.

